COMPUTER ENGINEERING
QUEZON CITY

Final Project: C.R.E.A.M. - Community Readied Executables & Application Marketplace

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

Item	Details
Project Title	C.R.E.A.M. - Community Readied Executables & Application Marketplace
Business Process Emulated	Digital Game Distribution and Social Networking Library System
Section / Group No.	CPE21S1 / Group 6
Instructor	Engr. Jimlord Quejado
Date Submitted	November 6, 2026
GitHub Repository	https://github.com/Pangilinancpe/Final-Project---DSA

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

General Objective:

To develop a C++ program that emulates the workings of a game hub software such as Steam created by VALVE. Including most of the processes of the application such as account creation, viewing of accounts, adding of games, viewing of games, posting comments/reviews, chatting, canvassing through game stores, and log in / log out system whilst applying the appropriate data structures and algorithms.

Specific Objectives:

1. To organize accounts, games, comments/reviews, and store data through header files with the application of pointer members and different methods.
2. To search through linked lists with integrated vectors with a variety of search techniques.
3. To utilize filtering through sorting algorithms with specified values for efficient canvassing.
4. To implement multi-user processing to fulfill real-time chatting with the use of multi-threading and File-Based Inter-Process Communication.
5. Be able to analyze the theoretical and experimental complexities of both time and space the functions take up.

COMPUTER ENGINEERING
QUEZON CITY

1. BUSINESS PROCESS

1.1 Organization and Business Process

This project is trying to emulate or to do the process of STEAM, the software or application that distributes and makes game softwares and publishes them on their own, and the system that allows the normal registered users to buy these game softwares and play with them. This software allows the Game developers for example to publish their own games, and allows to attached tags to these games to allow the normal users or the gamers to easily search the based on the tags that they have, for example a new game called The Wintering is tagged with [survival, mmorpg, and single player], by adding these tags, the users would be able to easily find the games that they want based on the tags that they enabled and easily put it onto their own carts.

1.2 Current Process (As-Is)

Step	Activity	Performed By	Data Involved
1	Users log in to the system	Users	UserID, Username, password - Account registration and logging in
2	Adding Funds to the Logged in User	Users	addFunds - banking system mockup
3	Adding Games to Cart	Users	addOwnedGame - adding games to the cart to be bought
4	Checking out the cart	Users	spendFunds, doCheckout, showUpdatedCart - Game buying
5	Download Game	Users	Game library / download execution and local library asset retrieval

1.3 Problems in the Current Process

1. The chat works between terminals on one computer that share a folder. It is not a network chat.
2. The wallet is simulated, and there are no refunds.
3. Passwords use a simple hash (djb2), which is fine for coursework but is not real security. The developer access code is also stored in the source code.
4. The cart, the install list, the history and the undo list live only in memory and are not saved between logins.
5. Downloads are a simulation: nothing is copied, only the progress is counted.

1.4 Proposed System

#	Feature	Business Task It Supports
1	Publishing Games	Product Catalog Management
2	Buying Games	Transaction Management
3	Wishing Games	Demand Forecasting & Lead Generation
4	User Game Feedback	Customer Satisfaction & Quality Assurance
5	User Messaging	Community Engagement & Customer Relationship Management (CRM)

1.5 Scope and Delimitations

The C.R.E.A.M system simulates the core functionalities of the STEAM by Valve which covers the organization of the created accounts in the software, games, and storing of the data inside the header files by the implementation of pointer members and different methods like (Binary Search Tree, Hash Table, and Singly-Linked List). Also, it covers the searching techniques to search through the linked list with their integrated vectors for the STEAM-like process itself. Moreover, the program uses filtering by the use of sorting algorithms that filter games in the game store by their values for utilizing the user's canvassing of games. Also, the program covers the implementation of the multi-user processing to initialize the multi-user interactivity or real-time chatting with other users to the program by the use of multi-threading.

The C.R.E.A.M system does not cover the multiple user logging in the system which means that another user cannot be logged in while another user is already logged in the system. Also, all the users can publish a game, which means that other users can flood the store by creating more games and we can't detect if the game is inappropriate to create. Moreover, the adding of funds process does not work by real-time adding of funds, it is just basically just accessing the wallet and then you add the money to your account at your own input. Lastly, we do not have the access to monitor the users in the real-time chatting feature. We can't monitor the chat if it's inappropriate, so we make the real-time chatting feature a "Freedom of Speech" to the users.

2. DATA STRUCTURES AND ALGORITHMS APPLIED

Course topics you may choose from: arrays and dynamic memory allocation; singly, doubly, and circular linked lists; stacks (array or linked-list based); queues (array or linked-list based); linear search; binary search; hashing; sorting algorithms; general trees and binary search trees; AVL trees; tree traversal algorithms; graphs (adjacency matrix or adjacency list); breadth-first search; depth-first search.

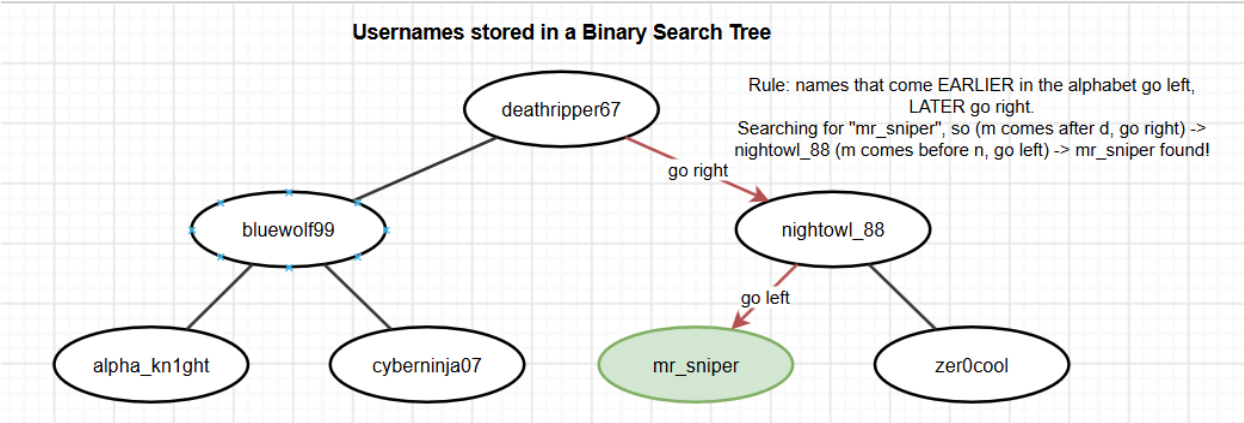
2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	**Binary Search Tree (BST)**	Looking up usernames during login/registration and exact store game title searches	Alphabetical insertion (accBstInsert, gameBstInsert), search lookup (accBstFind, gameBstFind), and tree deallocation (accBstFree, gameBstFree).
2	**Hash Table (HT)**	Hashing user passwords for secure authentication (account.h) and mapping chat sender IDs to unread message counts (chat.h).	djb2 hash accumulation (hashPassword), credential matching (loginAccount), and key-value lookup (std::map::find)
3	**Singly & Doubly Linked Lists (SLL & DLL)**	Primary linear storage for user accounts, game libraries, wishlists, shopping carts, reviews, and friends	Singly Linked Lists perform single-direction forward traversal, tail insertions, and single-pointer relinking deletions (such as showCart and removeFromCart), whereas Doubly Linked Lists enable bidirectional forward and backward traversal using two-way next and previous pointer updates for direct node insertion and decoupling.
4	**Queue integrated Linked List (QLL)**	First-In, First-Out (FIFO) shopping cart item handling and sequential order processing during checkout	Enqueue appending to cart tail (addToCart), sequential traversal (showCart), and dequeue purging upon checkout completion (clearCart)

5	**Vector (V)**	Buffering, sorting, and displaying recent message history during active chat sessions (chat.h)	Dynamic element appending (push/back), contiguous array indexing, and size tracking (size)

2.2 BINARY SEARCH TREE (BST)

Concept: The binary search tree is like a family tree like every of its node (person) can have at least two children, both left and right and now the rule in this one is the names that comes earlier, goes to the left and the one who comes later will go to the right, since we have this rule, it don't need to read every single one of the names that has been already listed. In the node we compare the name that we are looking for, then we go only in one way, it's either we go left or right just to find the name, we repeat this process until the name is found, if it is not found, that means the name is not in the tree or list.



Application in the System: In the system the BST is used for two things which are account username and game titles in the game store, if someone registers, the program searches the tree for its name and now if it finds it, the name is already taken. If it reaches an empty spot, the name is free and a new node is added right there, so the tree will always stay in alphabetical order and when someone logs in, the same search finds their account, the store works the same way the exact game titles searches.

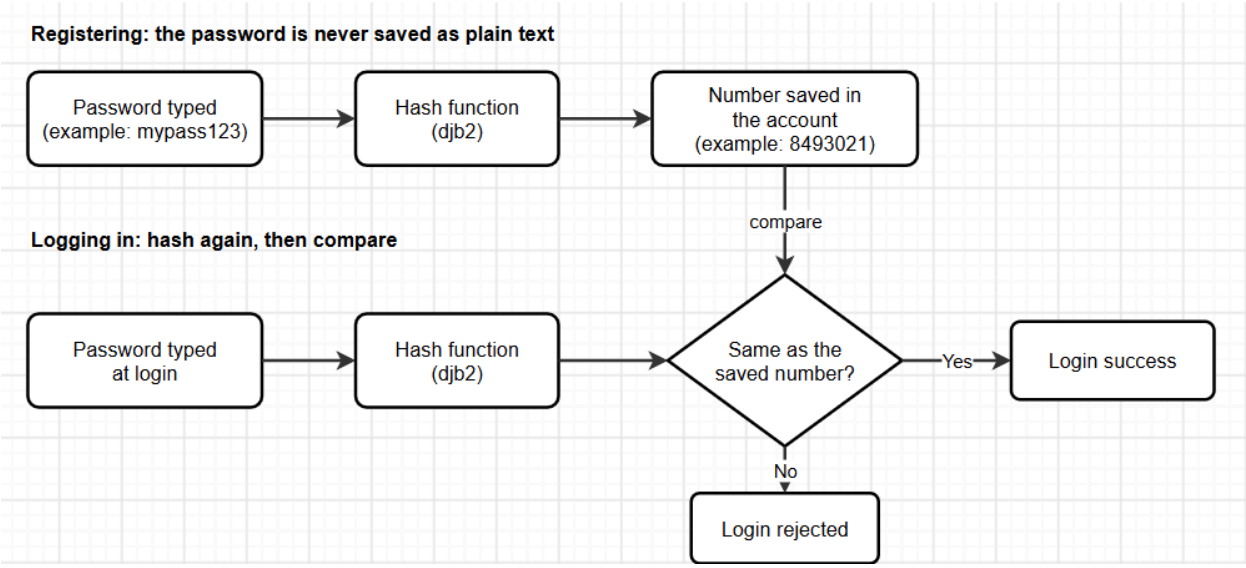
Justification: We use the BST since it is a BST, adding a name only needs one new node to be linked in. The tree also keeps the names in alphabetical order for free. The only weak point is that if names are added already sorted like a, b, c, d,then the tree turns into a long chain and becomes as slow as a list.

Operations and Complexity: Adding a name goes to O(log n), since we only go one way at each step, now searching a name goes to O(log n) too since they are the same.

2.3 HASH TABLE (HT)

Concept: The concept of a hash table is like a cabinet with a drawer and with each of the drawer is called a bucket, the bucket saves an information from created account, such as the username and its password then in order to decide which drawer use the program gives the item to the hash function then tells which drawer to open then the password turn into number so that the password will turn into number so that it wont save as a plain text, once the

player/user logs in, the program will turn the password into number again then it will check if it matches the save number, if the number matches then the user can log in.



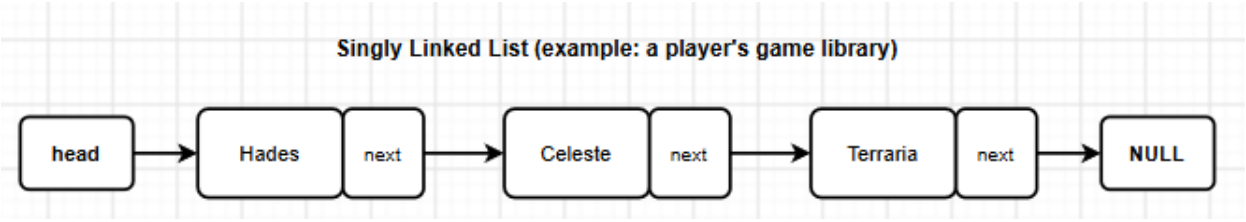
Application in the System:

Justification: The account has to be checked from the first account to the last, so every sign up, login it gets slower as the player database increases too, so this is where the hash table comes in, it fits better because on how it go straight to the bucket and the binary search tree give O(log n) and it will keep the accounts in sorted order, so the extra work is not worth it.

Operations and Complexity: In password it goes to O(1) since it is just a quick calculation now checking the login is also O(1).

2.4 SINGLY LINKED LIST (SLL)

Concept: The singly linked list is a chain of nodes that each nodes are holding some data and a pointer to the next node and the list remembers where the first node or the head node is and also the nodes can be anywhere in the memory, so the list does not need a fixed size and just simply grows by one node every time something is added and the last node points to nothing and that's the end of it.



Application in the System: Each game has its own linked list for its reviews, so every node holds the username of the reviewer and the review text. When a player posts a review, a new node is added at the end of that game's list, and when someone opens the game, the program starts at the head and goes node by node until the end to show all the reviews. The same idea is also used for the cart, like addToCart, ownsGame, removeFromCart and clearCart.

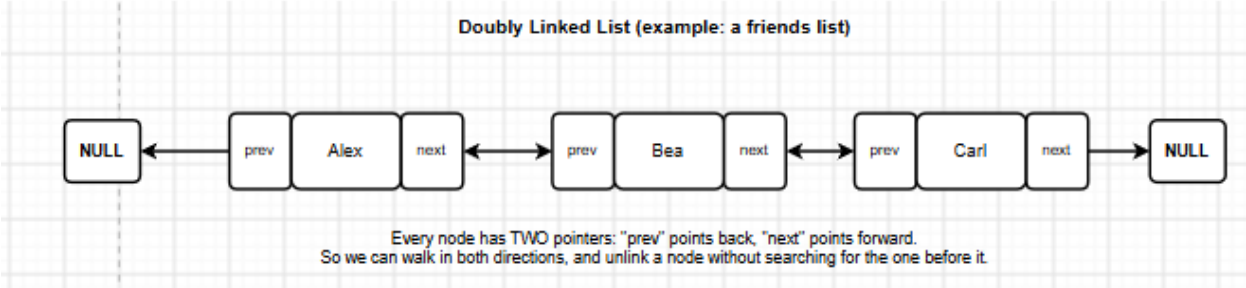
Justification: We use the SLL since we cannot know how many reviews a game will get, one game can have none and another can have a lot. An array needs a fixed size, so it would waste space or run out of it, but with the SLL we

just add one node when we need it and nothing is wasted. Also each node only has one pointer, so it uses less memory than the doubly linked list, and the reviews are shown in the same order they were posted anyway.

Operations and Complexity: Adding a node at the head or tail is O(1), since we already know where those are. Searching, like in ownsGame, is O(n) because we have to go node by node. Deleting a node is O(1) if we already have the node before it, like in removeFromCart, but if we still need to search for it first then it is O(n). Clearing the whole list, like clearCart, is O(n) too since every node has to be freed one by one. The space is O(n) with only one pointer for every node.

2.5 DOUBLY LINKED LIST (DLL)

Concept: The doubly linked list, it is kinda similar to singly linked list but this time each node has two pointers instead of one then the next which points to the node after it and the prev and which points to the node before it. So in short example its like a line of people, they know both who is in front and who is behind them and meaning they can walk forward and also backward.



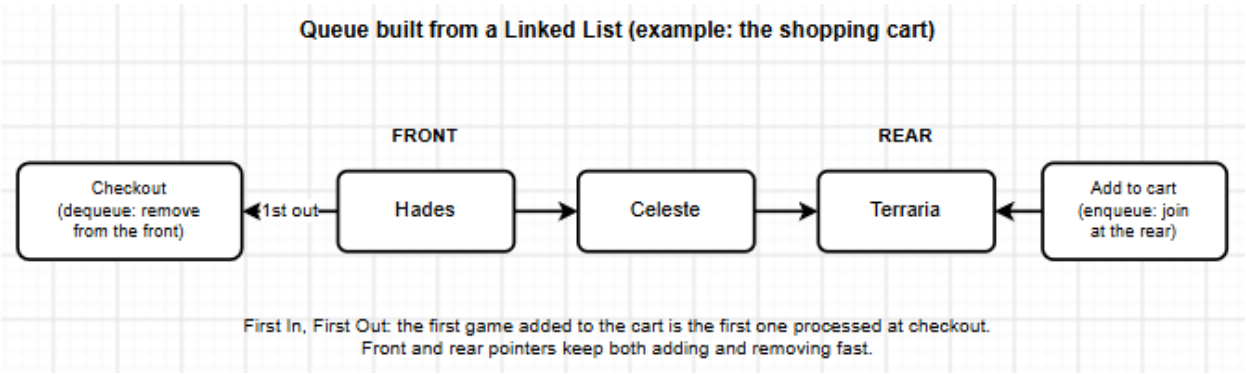
Application in the System: The system mostly uses singly linked lists with head and tail pointers, but the DLL would fit well in the parts where the user needs to go back and forth, like going to the next and previous page of the store catalog (showStorePage), undoing and redoing the changes in the cart (doCart), or scrolling up and down in the chat history (loadConversation). In these features each page or message would be a node, and the prev and next pointers let the program move one step in either direction.

Justification: With the SLL, going back one step means we have to start again from the head and walk all the way until we reach the node before it, and that is slow. With the DLL the prev pointer is already there so it goes straight back. The trade is that every node carries one extra pointer, but for features that move both ways it is worth it.

Operations and Complexity: Moving one step forward or backward is O(1), and going through the whole list in either direction is O(n). Inserting at a known position is O(1), and deleting a node is also O(1) without searching for the node before it, since the node already knows it. The space is O(n) with two pointers for every node.

2.6 QUEUE integrated LINKED LIST (QLL)

Concept: The queue is like the line in a cashier, the first one who lined up is the first one to be served, and this rule is called first in, first out. New ones always join at the back which is the rear, and the one being served leaves from the front. Since we build it using a linked list, the front is the head and the rear is the tail, so the queue can grow as long as it needs to without a fixed size.



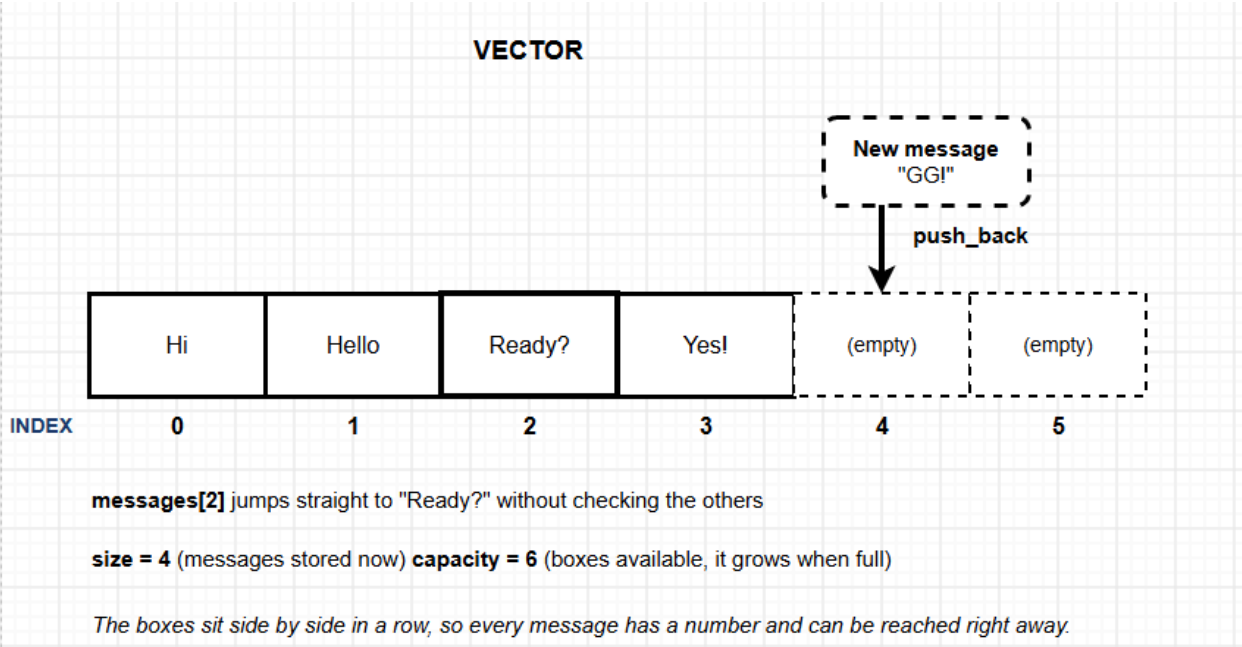
Application in the System: The queue is used for the game downloads. When a player downloads a game, a new node with the game information is added at the rear of the queue, and the downloads are done one at a time starting from the front. Once the game at the front is finished, it is removed from the queue and the next one in line becomes the front, and this goes on until the queue is empty.

Justification: We use a queue since the downloads should be fair, so the game that was requested first is the one that should finish first. An array would need us to move every item forward each time one is removed, but in the linked list queue we only change the head pointer, so it is faster and it also does not have a size limit.

Operations and Complexity: Enqueue is O(1) since we already have the pointer to the rear, dequeue is O(1) since we only move the head to the next node, and looking at the front is O(1) too. Going through the whole queue to display it is O(n), and the space is O(n).

2.7 VECTOR (V)

Concept: The Vector Algorithm is used as a dynamically-resizable array that automatically stores their elements in one block of memory and allows direct access using their index manages its own memory storage. Also, If there is no space left in the block, a new block of memory is allocated (usually twice the size of the previous block). The elements are copied across to the new block. The old block of memory is then deleted. This means that you don't need to worry about memory management as the vector will automatically resize. You also don't need to know how many elements you will have. In C++ vectors are implemented using (std::vector).



Application in the System: The Vector Algorithm is currently used in the buffering, sorting, and displaying of the messages history in the real-time chatting feature during active chat sessions which it is in (chat.h). First Step, the data (ChatMessage records) are being stored in the (chat.h) module which each ChatMessage records includes the message id, timestamp, sender id and name, receiver id, and the message text itself which are being processed in the loadConversation() function. Second Step, the function opens the shared chat file, where each line has the format (msgId|timestamp|fromId|fromName|toId|text), and reads it line by line in which each line is parsed into a ChatMessage by parseChatLine(), and malformed lines are skipped. Third Step, if the message was sent between user A and user B (either A to B or B to A), we add it to the end of our std::vector using the push_back method. Fourth Step, once the file has been read completely, the vector contains all the messages in chronological order since we can only add to the end of the file. Fifth Step, if the vector holds more than lastN messages, the oldest ones are erased from the front so that only the latest lastN remains. Sixth and last step, the vector is returned to the caller, which walks through it from index 0 to the end which displays each message with printChatMessage(), so the oldest message appears first and the newest last.

Justification: Provides efficient temporary buffering for dynamic message counts parsed line-by-line from chat.h. Contiguous memory allocation maximizes CPU cache locality during screen output, while dynamic capacity expansion (push_back) easily accommodates variable conversation lengths. Additionally, vector range iterators enable clean memory slicing (all.erase) to instantly trim the buffer down to the most recent messages prior to display.

Operations and Complexity: push_back is O(1) most of the time, and only when the block is full and has to be copied it takes O(n), but that rarely happens. Reading any message by its index is O(1). Erasing the old messages from the front is O(n) since the remaining ones have to shift forward, and going through all the messages to display them is also O(n). Reading and filtering the chat file is O(n) for the number of lines, and the space is O(n).

3. SYSTEM FLOWCHART

Symbol	Name	Use
Oval	Terminator	Start and End
Rectangle	Process	An action or computation, e.g., "Add patient to queue"
Parallelogram	Input / Output	Reading input or displaying output
Diamond	Decision	A yes/no question, e.g., "Is the queue empty?"
Rectangle with double sides	Predefined Process	An operation detailed in another flowchart
Arrow	Flow Line	The order of steps

3.1 Main System Flowchart

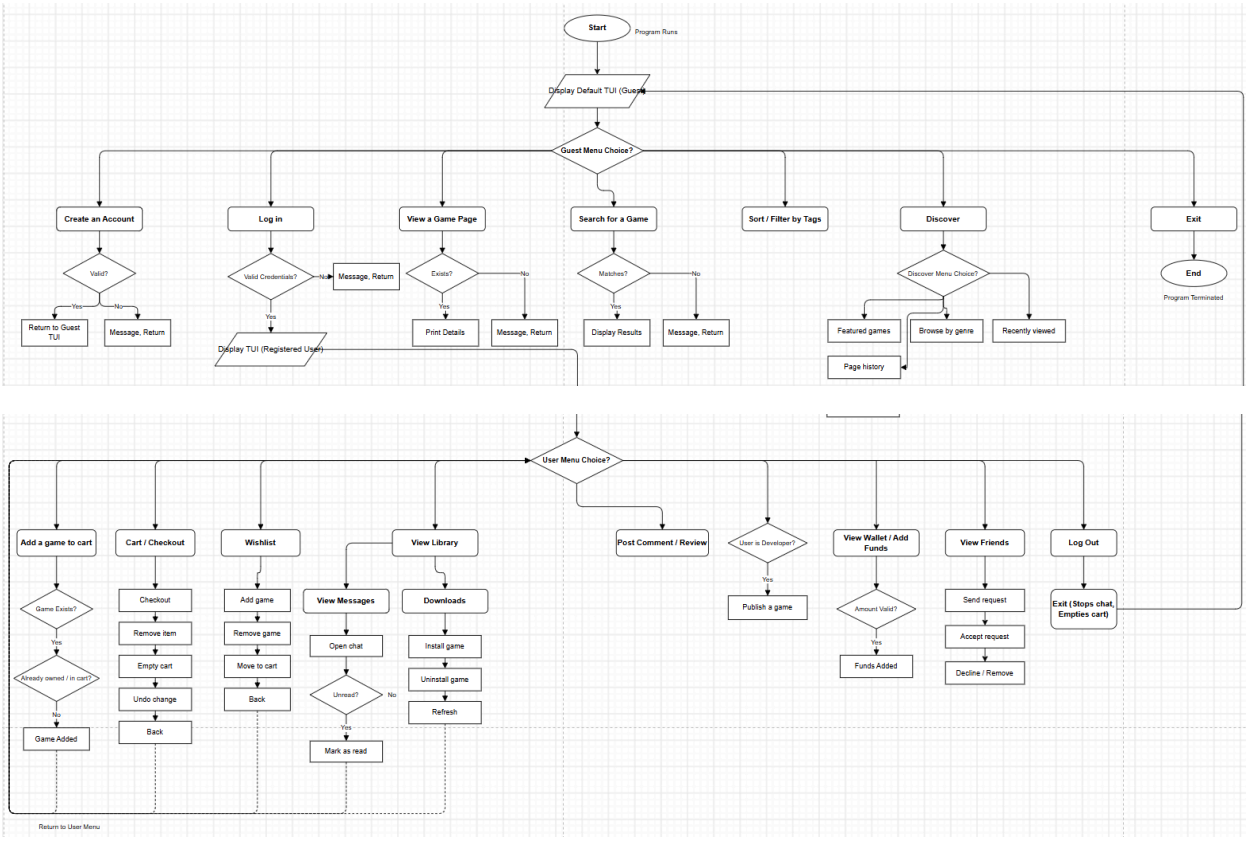


Figure 1. Flowchart of [Project Title]

3.2 Flow Description

1. The program runs and loads our store header file TUI.
2. The default TUI is our store page, browsing as a "guest user" with the given options; create an account (A), log in (B), view a game page (C), search for a game (D), and lastly exit the program (E).

A. Create an Account

1. The program will ask what your username will be with the set character limitations.
2. After that, it will ask for your password and a confirmation
3. Lastly, it'll promptly bring you back to the default TUI so you may log in.

B. Log In

1. The program will ask for your username
2. Then it will ask for the password
3. If all information is in the database, it will bring you to the store page browsing as a registered user with new options. If not, it will bring you back to the default TUI as a guest.
4. When logged in as a user, you are presented with the same options with new ones; Add a game to cart (a), cart/checkout (b), wishlist (c), viewing of your library (d), posting a comment/review (e), publish a game (f), viewing of your wallet or add funds (g), view friends (h), view messages (i), log out (j), and lastly exit.

    a. Adding a game to cart
        i. The program will ask for the game id or title
        ii. If the game exists, it will put it in the cart and you are brought back to the main menu.
        iii. If the game does not exist, it will print "Game not found" and have a suggestion of the closest title.
        iv. If you already own or have the game in your cart, it will inform you and bring you back to the store page.

    b. Cart / Checkout
        i. The program will display every game in your cart, the total price and your wallet balance.
        ii. It gives you 5 options: check out, remove an item, empty the cart, undo the last cart action, and go back to the store page.
        iii. For check out, if your wallet is not enough, it will print the total and your wallet balance and ask you to add funds. If you have enough money, it will be taken and the games in the cart will be added to your library and are removed from your cart. After that, a receipt will be printed.
        iv. For removing an item, it will ask for the game id or title and it will take it out of the cart.
        v. For emptying the cart, it will remove every game from the cart.

vi. When undoing the last cart change, it “undos” the most recent action you did (add or remove).

vii. When choosing to go back, it’ll bring you to the store page.

c. Wishlist

i. The program shows you the list of the games in your wishlist.

ii. It gives you the options to add a game, remove a game, move every game in the cart, or go back to store page

iii. When adding or removing the game, it will ask for the game IDand title. If you add an already owned game, it won’t be added. If you try adding a game that is already on the wishlist or removing a game that is not in the wishlist, will print a message to inform the user

iv. When moving everything into the cart, every game in the list that you do not own yet will be put in your cart, and the program will tell you.

v. When choosing to go back, it’ll bring you to the store page.

d. Viewing of your library

i. The program prints out your library’s name and how many games you own, and a list of all the games including their ID and title.

ii. If you do not have any games yet, it will print that the library is empty.

iii. Then it will bring you back to the store page.

e. Posting a comment / Review

i. The program will ask for the game ID or title.

ii. The rule here is that you can only review a game that you own, if you do not own the game, or if you are the publisher of this game, the program will inform you and bring you back to the store page

iii. It will ask if you recommend the game, then ask for your comment.

iv. If you have already reviewed the game, another post will overwrite your old review.

v. After that, it will print that the review has been posted or updated, then it will show on the game’s page

f. Publishing a Game (Developer accounts only)

i. This option only appears for accounts that were created as a developer.

ii. This will list all your published games with their respective price, number of owners, and the percentage of players who recommend it.

iii. It then gives you the option to publish a new game, edit one of your games, or go back to the store page.

iv. In publishing a new game, the program will show all the tags for the player to search for, then ask for the title, description, tags, and the price of the game (it can be free).

v. If the title is empty or already used by another game, there are no tags, or the price is invalid, it will inform you and you will fail to publish the game. If everything is in order, the game will be published and you will get a copy of it in your library, and it will print the game ID, price, and the tags.

vi. When editing one of your games, it will ask for the game ID or title (must be games you published), then it will ask for a new price, description, and tags. Leaving an input empty keeps the original value of the game.

vii. When choosing to go back, it will go back to the store page.

g. Viewing of your Wallet / Add funds

i. The program prints your current balance.

ii. It will ask for the amount to add in your wallet, and leaving it blank will cancel.

iii. The limit in adding money is 500, if it is outside the range or passes the wallet limit of 9,999.99, it will inform you and you will fail to add money.

iv. If valid, it prints the amount added and your new total balance, and you are automatically brought back to the store page.

h. Viewing friends

i. The program shows your friends (with the number of unread messages), the friend requests you have, and the ones you sent.

ii. It then gives you the option to send a friend request, accept a friend request, decline a request or remove a friend, or go back.

iii. For the first 3 options, the program will ask for the username of the other person, if they do not exist, the program will inform you.

iv. When sending, you cannot add yourself, you cannot send multiple requests to the same person, and if the person you are sending a request already sent you a friend request, both of you will automatically become friends.

v. When accepting, it will only work if the person has sent you a request.

vi. When declining or removing, it will disregard the request or remove the friendship.

vii. After each action, the friends list will show the updated version, and choosing to go back will bring you to the store page again.

i. Viewing messages

i. The program will print your friends list with the count of unreads, then gives you the option to open a chat, mark all messages as read, or go back.

ii. When opening a chat, it will ask for the username of a friend, you can only chat with friends.

iii. The chat log will show the recent 15 messages of your conversation. Type your message and press enter to send it. Messages from your friend appear on the screen live while the chat is open.

iv. When typing /exit, you will leave the chat and return to the messages screen.

v. When marking all messages as read, it will clear all the unread counts.

vi. When choosing to go back, it will bring you to the store page.

j. Download (install games)

i. The program shows the game that is currently installed with a progress bar, the games are waiting in the queue, and the games are already installed.

ii. It gives you the option to install a game from your library, uninstall a game, refresh the downloads, and to go back.

iii. When installing or uninstalling a game, it will ask for the game ID or title. You can only do this for your owned games. Installing puts the game in the download queue, and the games are installed one after the other in the background.

iv. When refreshing, it will update the progress.

v. Choosing to go back, will bring you to the store page

k. Bundle Optimizer (Wish List)

i. The program will go to your list of games in the wish list that you do not own yet. If it is empty, it will inform you and bring you back to the store page.

ii. It will ask for a budget (leaving it blank will use up all of your wallet balance).

iii. It then picks the combination of the games in your wishlist with the best total rating that still fits your budget, and shows the games and the total price.

iv. It asks if you want to add the games to your cart, entering yes will put the in your cart.

l. Friend Network

i. The program shows how many users are in the network and in your friends list, how many are your friends, friends of friends, and etc.

ii. It will list the people you may know with the number of mutual friends.

iii. It asks for a username to trace the connection to (putting in blank will go back). If there is a chain of friends, it prints the number of steps and names along the way. If not, it tells you there is no chain.

m. Log Out

i. The program will stop the chat, empty your cart, and log you out.

ii. It will print that you have logged out and bring you back to the store page browsing as a guest.

C. Viewing a Game Page as a Guest

1. The program will ask for either the game ID or title

2. If the game exists, it will print the game title, game description, tags, price, and reviews. If it does not exist, it will promptly bring you back to the default UI

D. Searching for a Game as a Guest

1. The program will ask for a title or tag that your desired game will have. It will also work for unfinished inputs, let's say half of a title.
2. After pressing enter, it will display all games that has the "term" or "tag" that you've entered and you'll be back to the default TUI

E. Sort / Filter by Tags (Guests and Registered Users)

1. The program will show the current sort order, the current tag filter, and every tag in the store with the number of games that use it.
2. It then gives you the option to choose the sort order, filter by tags, clear the filter, and go back to the store.
3. When choosing the sort order, it will list all the available sort orders and ask you to pick one. The store page, search results, and genre lists will follow the sorting order that you have chosen.
4. If you filter by tags, it will ask for the tags separated by commas. A game must have ALL the tags to be shown. If a tag is not used by a game, the program will inform you and the filter will not be applied.
5. By clearing the filter, it will show all the games again.
6. When choosing to go back, it will bring you back to the store page (with the chosen sort and filter if ever)

F. Discover (Guests and Registered User)

1. The program shows the menu which shows the featured games, browsed by genre, the recently viewed page history, algorithm lab, run all lessons, and the option to go back.
2. When choosing featured games, it will display the best rated games one at a time. Typing "n" for the next game, and typing p for the previous game (The list is in a circle).
3. When browsing by genre, it will show the genre tree with the list of games in each genre. You can display the games of a genre by typing the genre name, or show the tree in preorder, postorder, and level order.
4. In the recently viewed option, it will list the games of the pages you recently opened with the newest first.
5. In checking the page history, it will show the current game page. When typing "b", it will go back to the previous game page, and "f" to go forward, and another option to leave.
6. In the algorithm lab, it will compare the six sorting algorithms with the chosen amount of numbers and data shape, it compares linear and binary search, and compares fibonacci with plain recursion against memoization.
7. When running all lessons, it will run a short live demo of each lesson one step at a time. Pressing enter will continue to the next lesson, or an option to stop.
8. Choosing to go back will bring you back to the store page.

G. Exit.

1. For both Registered Users and Guests, Once picking the exit option, it will terminate the program.

4. OUTPUT

Feature	Figure No.	Input Used
Publishing Games (As a Developer)	Figure 2	Create an account, choose Developer (2), and enter the developer access code. Log in, open Developer Studio, and publish a game by entering its title, description, tags, and price.
Buying Games	Figure 3	Log in as a user, use Wallet to add funds, choose Add a game to the cart, then open Cart / checkout to pay.
Wishing Games	Figure 4	Log in as a user, add a game to your wishlist, then move the wishlist to the cart or run the Bundle optimizer.
User Game Feedback	Figure 5	Log in as a user who owns the game, choose Post a review / comment, select the game, pick recommend or not recommend, and type a comment.
User Messaging	Figure 6	Log in as a user who is already friends with another user, open the Messages menu, select the friend, and type a message. Open the same chat from a second terminal logged in as the friend to see it arrive live.
Invalid input handling	Figure 7	Enter an invalid input, such as a letter where a number is expected or a menu choice that doesn't exist. The program shows an error message and prompts again instead of crashing.

```
== CREATE ACCOUNT ================================================================
 (username: 3-20 letters, digits or _ ; password: at least 4 characters)
Username: Clear
Password: 67890
Confirm password: 67890
Account type: 1) Player  2) Developer [1]: 2
Developer access code: dev2026
  Developer account created! Your user id is 5. You can log in now.

+------------------------------------------------------------------------------+
|                           G A M E   S T O R E                                |
+------------------------------------------------------------------------------+
 Browsing as a guest. Create an account or log in to buy, review and chat.
--------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+--------------------+----------+-----------+----------------------+
| ST |  ID | TITLE              | PRICE    | RATING    | TAGS                 |
+----+-----+--------------------+----------+-----------+----------------------+
|    |   1 | Space Miner        | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash       | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley        | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders     | Free     | -         | sandbox, multipla... |
+----+-----+--------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 4 of 4 games

   1) Create account                  2) Log in
   3) View a game page                4) Search
   5) Sort / filter by tags           6) Discover (featured, genres, history, lab)
   0) Exit
```

```
== LOG IN ========================================================================
Username: Clear
Password: 67890
  Welcome back, Clear (developer)!

+------------------------------------------------------------------------------+
|                           G A M E   S T O R E                                |
+------------------------------------------------------------------------------+
 Clear  (id 5, DEVELOPER)
 Wallet $0.00  |  Cart 0  |  Wishlist 0  |  Unread 0  |  Friend requests 0
--------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+--------------------+----------+-----------+----------------------+
| ST |  ID | TITLE              | PRICE    | RATING    | TAGS                 |
+----+-----+--------------------+----------+-----------+----------------------+
|    |   1 | Space Miner        | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash       | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley        | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders     | Free     | -         | sandbox, multipla... |
+----+-----+--------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 4 of 4 games

   1) View a game page                2) Search
   3) Sort / filter by tags           4) Discover (featured, genres, history, lab)
   5) Add a game to the cart          6) Cart / checkout
   7) Wishlist                        8) My library
   9) Downloads (install games)      10) Bundle optimizer (wishlist)
  11) Post a review / comment        12) Wallet (add funds)
  13) Friends                        14) Friend network
  15) Messages                       16) Developer Studio
  17) Log out                         0) Exit
```



Figure 2. [Publishing Games (As a Developer)] - [Create an account, choose Developer (2), and enter the developer access code. Log in, open Developer Studio, and publish a game by entering its title, description, tags, and price]

```
== LOG IN ==========================================================================
Username: Yessir
Password: 1234567
  Welcome back, Yessir!

+-------------------------------------------------------------------------------+
|                            G A M E   S T O R E                                |
+-------------------------------------------------------------------------------+
 Yessir  (id 3, PLAYER)
 Wallet $85.01  |  Cart 0  |  Wishlist 0  |  Unread 0  |  Friend requests 0
-------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+--------------------+----------+-----------+----------------------+
| ST |  ID | TITLE              | PRICE    | RATING    | TAGS                 |
+----+-----+--------------------+----------+-----------+----------------------+
| O  |   1 | Space Miner        | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash       | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley        | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders     | Free     | -         | sandbox, multipla... |
+----+-----+--------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 4 of 4 games

   1) View a game page                  2) Search
   3) Sort / filter by tags             4) Discover (featured, genres, history, lab)
   5) Add a game to the cart            6) Cart / checkout
   7) Wishlist                          8) My library
   9) Downloads (install games)        10) Bundle optimizer (wishlist)
  11) Post a review / comment          12) Wallet (add funds)
  13) Friends                          14) Friend network
  15) Messages                         16) Log out
   0) Exit
```

```
  Added $14.99. New balance: $100.00

+-------------------------------------------------------------------------------+
|                            G A M E   S T O R E                                |
+-------------------------------------------------------------------------------+
 Yessir  (id 3, PLAYER)
 Wallet $100.00  |  Cart 0  |  Wishlist 0  |  Unread 0  |  Friend requests 0
-------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+--------------------+----------+-----------+----------------------+
| ST |  ID | TITLE              | PRICE    | RATING    | TAGS                 |
+----+-----+--------------------+----------+-----------+----------------------+
| O  |   1 | Space Miner        | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash       | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley        | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders     | Free     | -         | sandbox, multipla... |
+----+-----+--------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 4 of 4 games

   1) View a game page                  2) Search
   3) Sort / filter by tags             4) Discover (featured, genres, history, lab)
   5) Add a game to the cart            6) Cart / checkout
   7) Wishlist                          8) My library
   9) Downloads (install games)        10) Bundle optimizer (wishlist)
  11) Post a review / comment          12) Wallet (add funds)
  13) Friends                          14) Friend network
  15) Messages                         16) Log out
   0) Exit
```

```
-- Receipt --
  Space Miner  $14.99
  Paid: $14.99   Balance left: $85.01

+------------------------------------------------------------------------------+
|                          G A M E   S T O R E                                 |
+------------------------------------------------------------------------------+
 Yessir  (id 3, PLAYER)
 Wallet $85.01  |  Cart 0  |  Wishlist 0  |  Unread 0  |  Friend requests 0
------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+-------------------+----------+-----------+----------------------+
| ST |  ID | TITLE             | PRICE    | RATING    | TAGS                 |
+----+-----+-------------------+----------+-----------+----------------------+
| O  |   1 | Space Miner       | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash      | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley       | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders    | Free     | -         | sandbox, multipla... |
+----+-----+-------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 4 of 4 games

   1) View a game page                 2) Search
   3) Sort / filter by tags            4) Discover (featured, genres, history, lab)
   5) Add a game to the cart           6) Cart / checkout
   7) Wishlist                         8) My library
   9) Downloads (install games)       10) Bundle optimizer (wishlist)
  11) Post a review / comment         12) Wallet (add funds)
  13) Friends                         14) Friend network
  15) Messages                        16) Log out
   0) Exit
```

Figure 3. [Buying Games] - [Log in as a user, use Wallet to add funds, choose Add a game to the cart, then open Cart / checkout to pay]

```
+------------------------------------------------------------------------------+
|                          G A M E   S T O R E                                 |
+------------------------------------------------------------------------------+
 Yessir  (id 3, PLAYER)
 Wallet $100.00  |  Cart 0  |  Wishlist 0  |  Unread 0  |  Friend requests 0
------------------------------------------------------------------------------
 Sort: Newest last (default)
+----+-----+-------------------+----------+-----------+----------------------+
| ST |  ID | TITLE             | PRICE    | RATING    | TAGS                 |
+----+-----+-------------------+----------+-----------+----------------------+
| O  |   1 | Space Miner       | $14.99   | -         | simulation, space... |
|    |   2 | Dungeon Dash      | $9.99    | -         | action, roguelike    |
|    |   3 | Farm Valley       | $19.99   | -         | farming, relaxing... |
|    |   4 | Block Builders    | Free     | -         | sandbox, multipla... |
|    |   5 | Starshooter X     | $10.99   | -         | shooting, action,... |
+----+-----+-------------------+----------+-----------+----------------------+
 ST: O owned, C in cart, W wishlist   Showing 5 of 5 games

   1) View a game page                 2) Search
   3) Sort / filter by tags            4) Discover (featured, genres, history, lab)
   5) Add a game to the cart           6) Cart / checkout
   7) Wishlist                         8) My library
   9) Downloads (install games)       10) Bundle optimizer (wishlist)
  11) Post a review / comment         12) Wallet (add funds)
  13) Friends                         14) Friend network
  15) Messages                        16) Log out
   0) Exit
```

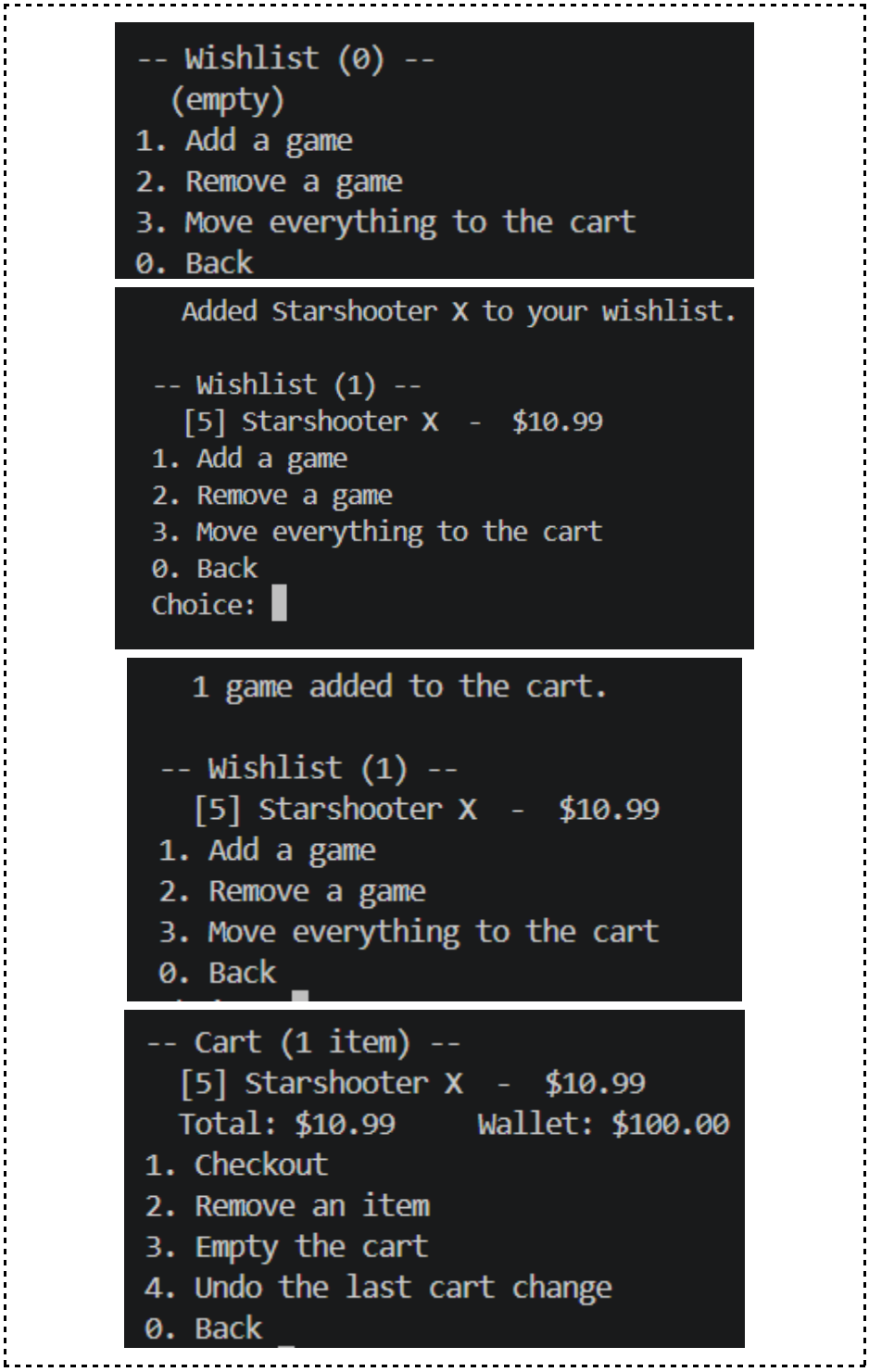


Figure 4. [Wishing Games] - [Log in as a user, add a game to your wishlist, then move the wishlist to the cart or run the Bundle optimizer]

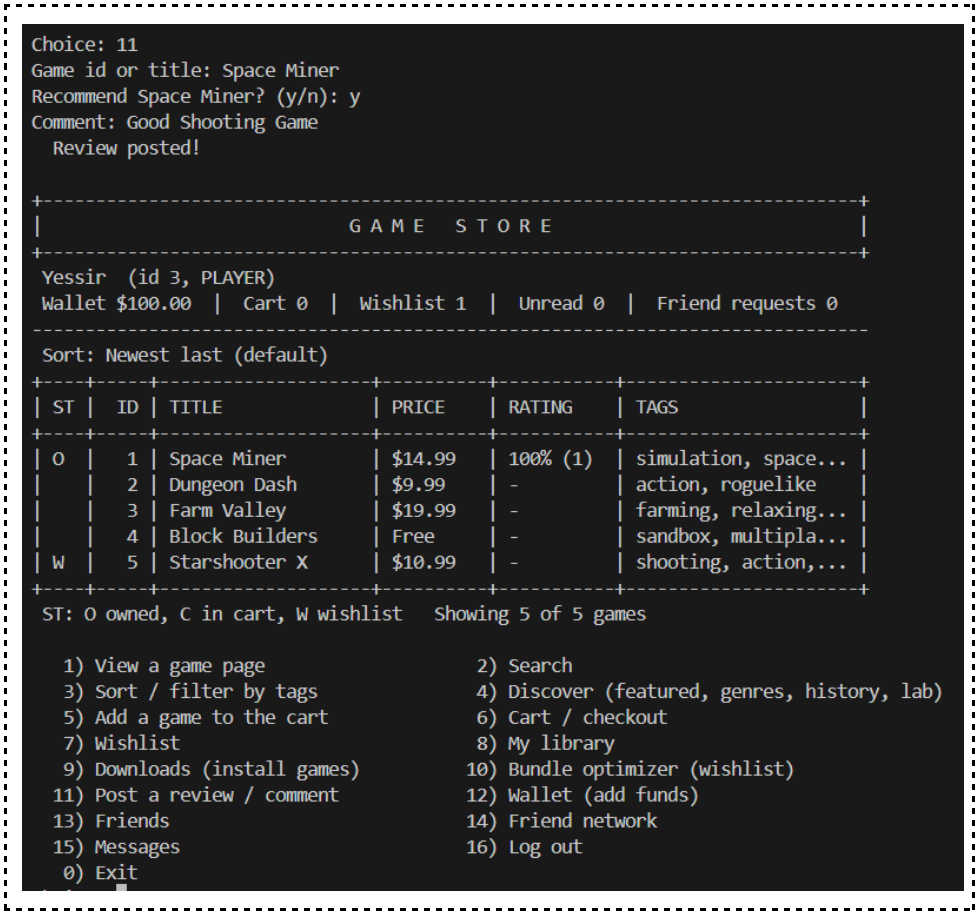
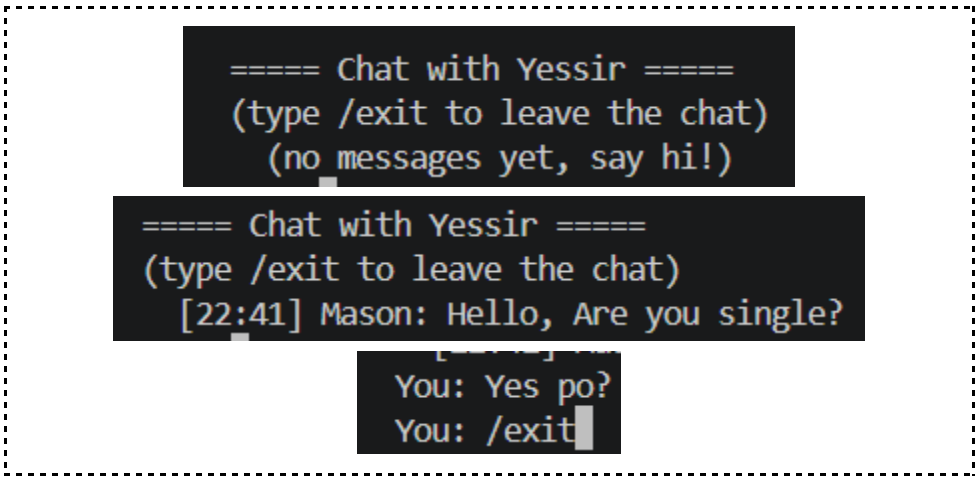


Figure 5. [User-Game Feedback] - [Log in as a user who owns the game, choose Post a review / comment, select the game, pick recommend or not recommend, and type a comment.]



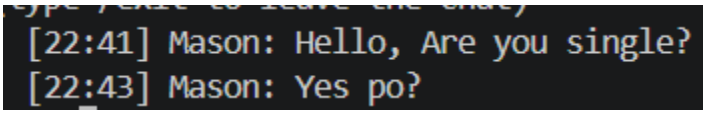


Figure 6. [User Messaging] - [Log in as a user who is already friends with another user, open the Messages menu, select the friend, and type a message. Open the same chat from a second terminal logged in as the friend to see it arrive live.]



Figure 7. [Invalid input handling] - [Enter an invalid input, such as a letter where a number is expected or a menu choice that doesn't exist. The program shows an error message and prompts again instead of crashing.]

5. PERFORMANCE ANALYSIS

5.1 Theoretical Analysis

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
Add game or record at the head	Singly linked list	O(1)	O(1)	O(1)	O(1)
Find a record by position or value	Singly linked list (linear search)	O(1)	O(n)	O(n)	O(1)
Find a game by title or a user by username	Binary search tree	O(1)	O(log n)	O(n)	O(1)
Insert a game or user	Binary search tree	O(1)	O(log n)	O(n)	O(1)
Find a game in a sorted array	Binary search	O(1)	O(log n)	O(log n)	O(1)
Sort the store page	Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)

Derivation:

Bubble sort

- The outer loop runs at most n-1 passes. Pass i makes n-i comparisons.
- Total comparisons are (n-1) + (n-2) + ... + 1 = n(n-1)/2, which is $O(n^2)$.
- If a pass has no swap, the loop stops. On sorted input only the first pass runs, with n-1 comparisons, so the best case is O(n).
- It sorts in place with a few extra variables, so the space is O(1).
- Measured with n = 10,000 random numbers: 49,994,054 comparisons. The formula gives 49,995,000. Sorted input took 9,999 comparisons, which is n-1.

Binary search.

- Each step compares the middle item and discards half of the range. After k steps, n/2^k items remain.

- The search stops when n/2^k reaches 1, so k = $\log_2$ n and the time is O(log n).
- The best case is O(1), when the middle item matches at once.
- It keeps only the low and high indexes, so the space is O(1).
- Measured with n = 1,000,000 sorted numbers: 21 comparisons. $\log_2$ 1,000,000 is about 19.9. Linear search took 1,000,000.

5.2 Experimental Analysis

Setup Item	Details
Computer (CPU, RAM)	AMD Ryzen 9 AI 365, 16gb Ram
Operating system	Windows 11 Pro 26H2
Compiler and settings	C++17
Timing method	std::chrono::steady_clock
Input data	Randomly generated integer records using a fixed random seed
Repetitions	1 run per input size per algorithm

Input Size (n)	[Operation 1] Time (ms)	[Operation 2] Time (ms)	[Operation 3] Time (ms)
100	0.006	0.018	0.006
1,000	0.081	0.083	0.07
10,000	1.386	1.002	0.919
100,000	19.160	11.943	10.415

100 numbers, Random

Algorithm	Time(ms)	Compares	Moves	Stable	Average	Worst
Bubble Sort	0.022	4929	2366	yes	O(n^2)	O(n^2)
Selection Sort	0.013	4950	96	no	O(n^2)	O(n^2)
Insertion Sort	0.007	2464	2460	yes	O(n^2)	O(n^2)
Shell Sort	0.006	903	684	no	O(n^1.5)	O(n^2)
Merge Sort	0.018	542	672	yes	O(n log n)	O(n log n)
Quick Sort	0.006	795	162	no	O(n log n)	O(n^2)

Figure 5.1 Running time vs. input size for [100]

1000 numbers, Random

Algorithm	Time(ms)	Compares	Moves	Stable	Average	Worst
Bubble Sort	1.659	496419	246782	yes	O(n^2)	O(n^2)
Selection Sort	1.769	499500	992	no	O(n^2)	O(n^2)
Insertion Sort	1.079	247779	247773	yes	O(n^2)	O(n^2)
Shell Sort	0.081	14701	11118	no	O(n^1.5)	O(n^2)
Merge Sort	0.083	8695	9976	yes	O(n log n)	O(n log n)
Quick Sort	0.070	13775	2246	no	O(n log n)	O(n^2)

Figure 5.2 Running time vs. input size for [1000]

10000 numbers, Random

Algorithm	Time(ms)	Compares	Moves	Stable	Average	Worst
Bubble Sort	179.757	49994054	25074455	yes	O(n^2)	O(n^2)
Selection Sort	99.348	49995000	9986	no	O(n^2)	O(n^2)
Insertion Sort	64.549	25084451	25084443	yes	O(n^2)	O(n^2)
Shell Sort	1.386	268371	211938	no	O(n^1.5)	O(n^2)
Merge Sort	1.002	120353	133616	yes	O(n log n)	O(n log n)
Quick Sort	0.919	184693	30749	no	O(n log n)	O(n^2)

Figure 5.3 Running time vs. input size for [10000]

```
100000 numbers, Random
 Algorithm        Time(ms)   Compares        Moves  Stable  Average      Worst
 Bubble Sort      skipped (a quadratic sort would take too long above 30000)
 Selection Sort   skipped (a quadratic sort would take too long above 30000)
 Insertion Sort   skipped (a quadratic sort would take too long above 30000)
 Shell Sort        19.160    4312869      3635019  no      O(n^1.5)    O(n^2)
 Merge Sort        11.943    1536134      1668928  yes     O(n log n)  O(n log n)
 Quick Sort        10.415    2311986       385119  no      O(n log n)  O(n^2)
```

Figure 5.4 Running time vs. input size for [100000]

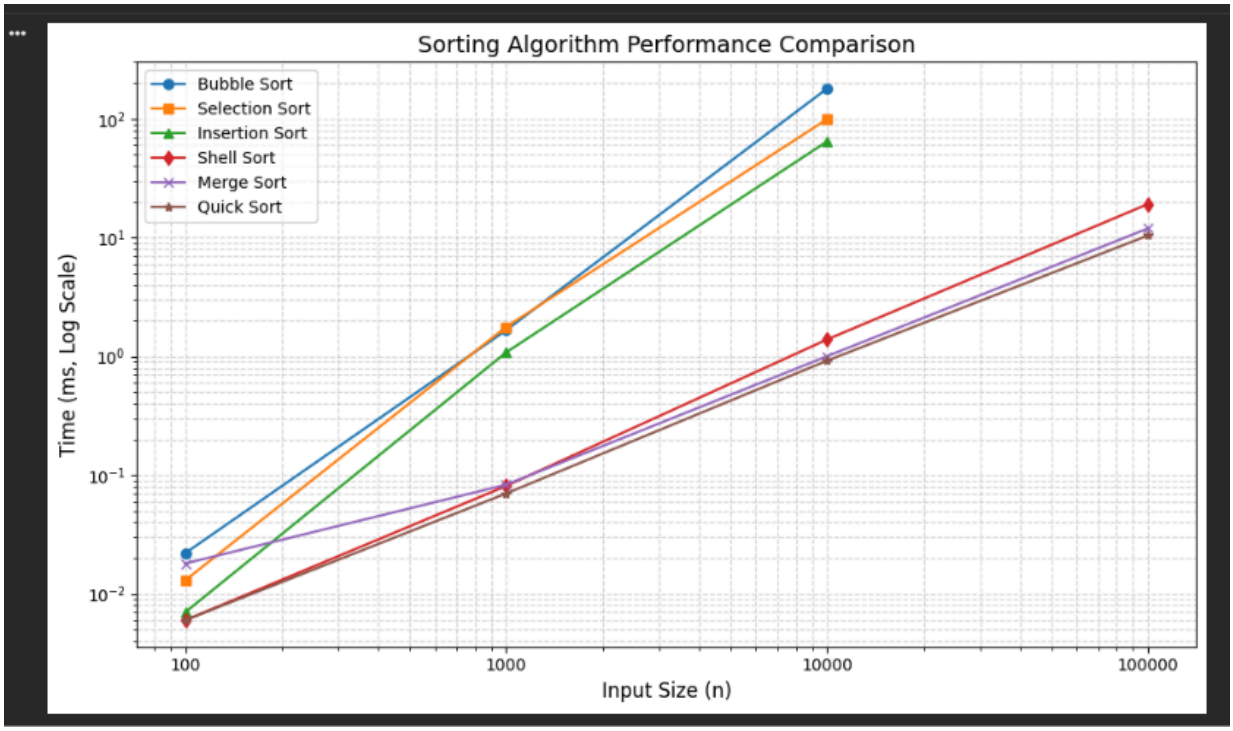


Figure 5.5: Line Graph of time taken per operations of the 6 Sorting algo

5.3 Comparison of Theoretical and Experimental Results

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
Bubble Sort	$O(n^2)$	Comparisons increase quadratically as input size increases.	Yes

Selection Sort	$O(n^2)$	Comparisons increase quadratically as input size increases.	Yes
Insertion Sort	$O(n^2)$	Performance depends on the initial arrangement of elements.	Yes
Shell Sort	$O(n\log n)$	Growth varies depending on the gap sequence and input arrangement.	No
Merge Sort	$O(n \log n)$	Expected to scale efficiently as input size increases.	Yes
Quick Sort	$O(n \log n)$	Expected to scale efficiently on random input; worst case is O(n^2).	No

The theoretical and experimental results show the differences in efficiency among the algorithms used in the Game Store application.

For sorting algorithms Bubble Sort, Selection Sort and Insertion Sort usually have quadratic time complexity. This means they are not very efficient when dealing with datasets. On the other hand , Merge Sort and Quick Sort have an average time complexity of O(n log n). This allows them to handle datasets much better. Shell Sort also does better than the quadratic methods but its performance depends on the gap sequence that is used.

For searching algorithms Linear Search has a time complexity of O(n). It may need to check every element until it finds the target. Binary Search has a time complexity of O(log n). This makes it more efficient for datasets because it keeps cutting the search area in half.

The Fibonacci comparison also shows the benefit of using optimization techniques. Plain recursion leads to repeated calculations. This causes the time complexity to grow exponentially. Memoization saves the results of calculations. This reduces the time complexity to O(n).

Even though theoretical complexities explain the expected performance of each algorithm the actual execution times can vary. This happens because of differences in hardware, memory access, caching, compiler optimizations and other processes running on the system.

Overall the results from the Algorithm Lab show how the choice of algorithm and the use of optimization techniques affect efficiency. The theoretical analysis helps explain the performance differences seen in the program. This is especially clear when comparing algorithms, with time complexities.

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The C.R.E.A.M. The application is effective in replicating the working process of the Steam platform by allowing users to register accounts, authenticate credentials, manage game collection, operate session shopping cart, track wishlists, write reviews, and communicate with each other. All of the key goals have been achieved with the help of creating custom data structures and writing the corresponding C++ code using Binary Search Trees (BST) for efficient alphabetization of users' names and titles of games, djb2 hashing for authentication, Singly and Doubly Linked Lists for managing user's items and cart, Queue-based lists for order execution, and dynamic vectors for message buffering. Theory-based complexity calculations prove the optimal operation of the algorithm with complexity O(log n) for searching in trees and O(1) for hash checking, whereas practical execution proves Big-O efficiency predictions on growing inputs.

6.2 Recommendations

- **Self-balancing search trees are something that might be considered in order to improve stability**: Moving from the simple Binary Search Tree to the AVL or Red-Black tree will ensure search operations' speed and prevent the degradation of performance into linear O(n) due to inserting elements alphabetically.

- **Persistent database configuration would definitely help with session handling**: Moving from in-memory linked list data structures to SQLite database will allow keeping user shopping cart, wishlist, and installed games persistent between application s**essions.**

- **Considering rethinking the security approach would greatly increase the security of the app**: Using modern hashing algorithms such as SHA-256 in order to replace outdated djb2 algorithms, as well as removing all developer codes from C++ would allow protecting user's credentials.

- **Moving to network sockets would greatly improve messaging service**: Moving from the file-based chat to using TCP/IP sockets will allow messaging in real time even if users are working on separate machines.

7. REFERENCES

Steam Store. (n.d.). https://store.steampowered.com/

MendozaRoycpe/CPE010-Data-Structures-and-Algorithms: A simple repository for my CPE 010: Data Structures and Algorithms course in Technological Institute of the Philippines AY. 2026-2027 First Semester. (n.d.). https://github.com/MendozaRoycpe/CPE010-Data-Structures-and-Algorithms

APPENDICES

Appendix A. Source Code

```
//============================================================================
// FUNCTION INDEX of main.cpp  (generated by tools/docgen.py, do not edit by hand)
//   line  function                                kind
//     45  wipeConsole                             function
//     75  isScreenChangingPrompt                  function
//     97  ScreenBuf::releaseHeld                  method
//    108  ScreenBuf::write                        method
//    127  ScreenBuf::ScreenBuf                    constructor
//    134  ScreenBuf::userPressedEnter             method
//    146  ScreenBuf::clearNow                     method
//    155  ScreenBuf::finish                       method
//    168  ScreenBuf::overflow                     method
//    181  ScreenBuf::xsputn                       method
//    194  ScreenBuf::sync                         method
//    219  LineBuf::LineBuf                        constructor
//    228  LineBuf::underflow                      method
//    250  isRealTerminal                          function
//    265  main                                    function
//============================================================================

#include "store.h"
#include <streambuf>
#include <mutex>

// options:  --plain = no colors     --color = force colors
//============================================================================
// CLEAR SCREEN
// Clears the terminal so the old menus don't stay on screen.
//
// Simple idea:
//   1. The user answers a menu ("Choice: ")  ->  clear the screen, then show the result.
//   2. The program asks for more info ("Username: ")  ->  don't clear, the user still needs it.
//============================================================================

// clears the terminal
// --clear: some consoles (for example the CLion Run window) are not real terminals and ignore the
clear codes.
// With this flag the screen is also pushed away with blank lines, so only the new content stays in
view.
static bool g_floodBlankLines = false;

//--- wipeConsole ------------------------------------------------------------------------------
// What    : Clears the screen and the scrollback (ANSI codes, or the Windows console API). With
//           --clear it also prints blank lines, for consoles that ignore the clear codes.
// Used by : ScreenBuf::clearNow, ScreenBuf::write
//----------------------------------------------------------------------------------------------
void wipeConsole(std::streambuf* dest) {
```

```
    dest->pubsync();
#ifdef _WIN32
    HANDLE hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    if (hStdOut != INVALID_HANDLE_VALUE && GetConsoleScreenBufferInfo(hStdOut, &csbi)) {
        DWORD count;
        DWORD cellCount = (DWORD)csbi.dwSize.X * (DWORD)csbi.dwSize.Y;
        COORD homeCoords = {0, 0};
        FillConsoleOutputCharacterA(hStdOut, ' ', cellCount, homeCoords, &count);
        FillConsoleOutputAttribute(hStdOut, csbi.wAttributes, cellCount, homeCoords, &count);
        SetConsoleCursorPosition(hStdOut, homeCoords);
    }
#endif
    const char codes[] = "\033[2J\033[3J\033[H";
    dest->sputn(codes, (std::streamsize)(sizeof(codes) - 1));
    if (g_floodBlankLines) {
        const std::string blanks(80, '\n');
        dest->sputn(blanks.data(), (std::streamsize)blanks.size());
    }
    dest->pubsync();
}

// prompts after which the user is looking at a NEW screen
//--- isScreenChangingPrompt -------------------------------------------------------------
// What    : Tells if a prompt belongs to a screen that should be cleared when the user answers it
//           ("Choice: ", "Search (", ...).
// Returns : true for those prompts
// Used by : ScreenBuf::userPressedEnter
//----------------------------------------------------------------------------------------------
bool isScreenChangingPrompt(const std::string& p) {
    const char* starts[] = {"Choice: ", "Game id or title: ", "Search (", "Amount to add", "Chat with
(", "Sort by: "};
    for (size_t i = 0; i < sizeof(starts) / sizeof(starts[0]); i++) {
        if (p.compare(0, std::string(starts[i]).size(), starts[i]) == 0) return true;
    }
    return false;
}

// filter on std::cout
class ScreenBuf : public std::streambuf {
private:
    std::streambuf* dest;
    std::mutex lock;              // the chat thread prints too
    std::string line;             // the unfinished line printed so far (a prompt, while the program
waits)
    std::string prompt;           // the last prompt that was flushed
    bool pendingClear = false;    // the user moved on, clear before the next real output
    std::string held;             // output that is waiting while pendingClear is true
```

```
//--- ScreenBuf::releaseHeld ------------------------------------------------------------------------
// What    : Sends the output that was held back to the real console.
// Used by : ScreenBuf::finish, ScreenBuf::sync, ScreenBuf::write
//-----------------------------------------------------------------------------------------------------
    void releaseHeld() {
        if (!held.empty()) dest->sputn(held.data(), (std::streamsize)held.size());
        held.clear();
    }

//--- ScreenBuf::write ------------------------------------------------------------------------------
// What    : Common writer of overflow and xsputn. It keeps track of the unfinished line (the
//           prompt). When a clear is pending it holds the output until a full line arrives, then
//           clears the screen and releases it.
// Used by : ScreenBuf::overflow, ScreenBuf::xsputn
//-----------------------------------------------------------------------------------------------------
    void write(const char* s, std::streamsize n) {
        for (std::streamsize i = 0; i < n; i++) {
            if (s[i] == '\n') line.clear();
            else line.push_back(s[i]);
        }
        if (!pendingClear) { dest->sputn(s, n); return; }

        held.append(s, (size_t)n);
        if (held.find('\n') != std::string::npos) {     // a result or a new screen is coming: clear
first
            pendingClear = false;
            wipeConsole(dest);
            releaseHeld();
        }
    }
public:
//--- ScreenBuf::ScreenBuf --------------------------------------------------------------------------
// What    : Wraps the output stream buffer that receives the text.
// Used by : main
//-----------------------------------------------------------------------------------------------------
    explicit ScreenBuf(std::streambuf* target) : dest(target) {}
    // called by LineBuf when the user pressed Enter
//--- ScreenBuf::userPressedEnter -------------------------------------------------------------------
// What    : Called by LineBuf when the user pressed Enter. If the last prompt was a screen-changing
//           one, the next output starts on a cleared screen.
// Used by : LineBuf::underflow
//-----------------------------------------------------------------------------------------------------
    void userPressedEnter() {
        std::lock_guard<std::mutex> guard(lock);
        bool changed = isScreenChangingPrompt(prompt);
        prompt.clear();
        line.clear();
```

```
        if (changed) pendingClear = true;
    }
    // clear right now (used once at the start)
//--- ScreenBuf::clearNow -----------------------------------------------------------------------
// What    : Clears the console right away (done once at the start).
// Used by : main
//-----------------------------------------------------------------------------------------------
    void clearNow() {
        std::lock_guard<std::mutex> guard(lock);
        wipeConsole(dest);
    }
    // give back anything that is still waiting
//--- ScreenBuf::finish -------------------------------------------------------------------------
// What    : Cancels a pending clear and prints whatever is still held, at the end of the program.
// Used by : main
//-----------------------------------------------------------------------------------------------
    void finish() {
        std::lock_guard<std::mutex> guard(lock);
        pendingClear = false;
        releaseHeld();
        dest->pubsync();
    }

protected:
//--- ScreenBuf::overflow -----------------------------------------------------------------------
// What    : std::streambuf hook for one character.
// Returns : the character, or EOF on error
// Used by : (nothing yet: not called from the program)
//-----------------------------------------------------------------------------------------------
    int_type overflow(int_type ch) override {
        if (traits_type::eq_int_type(ch, traits_type::eof())) return traits_type::not_eof(ch);
        char c = traits_type::to_char_type(ch);
        std::lock_guard<std::mutex> guard(lock);
        write(&c, 1);
        return ch;
    }

//--- ScreenBuf::xsputn -------------------------------------------------------------------------
// What    : std::streambuf hook for a block of characters.
// Returns : how many characters were taken
// Used by : (nothing yet: not called from the program)
//-----------------------------------------------------------------------------------------------
    std::streamsize xsputn(const char* s, std::streamsize n) override {
        std::lock_guard<std::mutex> guard(lock);
        write(s, n);
        return n;
    }
```

```
//--- ScreenBuf::sync -----------------------------------------------------------------------
// What    : Flush hook. A flush with an unfinished line is a prompt, so a pending clear is
//           cancelled (the user still needs the old screen to answer) and the prompt text is
//           remembered.
// Returns : 0 on success
// Used by : (nothing yet: not called from the program)
//-----------------------------------------------------------------------------------------------
    int sync() override {
        std::lock_guard<std::mutex> guard(lock);
        // a flush with unfinished text and nothing else is a prompt (the program waits for the user).
        // the user still needs the old screen to answer it, so cancel the clear.
        if (pendingClear && !held.empty() && held.find('\n') == std::string::npos) {
            pendingClear = false;
            releaseHeld();
        }
        if (!line.empty()) prompt = line;
        return dest->pubsync();
    }
};

// filter on std::cin: hands over one line at a time and tells ScreenBuf when a line was entered
class LineBuf : public std::streambuf {
private:
    std::streambuf* src;
    ScreenBuf* screen;
    std::string buffer;

public:
//--- LineBuf::LineBuf ----------------------------------------------------------------------
// What    : Wraps the input stream buffer and the ScreenBuf.
// Used by : main
//-----------------------------------------------------------------------------------------------
    LineBuf(std::streambuf* source, ScreenBuf* out) : src(source), screen(out) {}

protected:
//--- LineBuf::underflow --------------------------------------------------------------------
// What    : std::streambuf hook for input: reads one whole line from the real input, tells the
//           ScreenBuf that Enter was pressed, and hands the line over.
// Returns : the next character, or EOF at the end of the input
// Used by : (nothing yet: not called from the program)
//-----------------------------------------------------------------------------------------------
    int_type underflow() override {
        if (gptr() < egptr()) return traits_type::to_int_type(*gptr());

        buffer.clear();
        int_type c;
        while (!traits_type::eq_int_type(c = src->sbumpc(), traits_type::eof())) {
            buffer.push_back(traits_type::to_char_type(c));
```

```
            if (buffer.back() == '\n') break;
        }
        if (buffer.empty()) return traits_type::eof();

        screen->userPressedEnter();
        setg(&buffer[0], &buffer[0], &buffer[0] + buffer.size());
        return traits_type::to_int_type(*gptr());
    }
};

//--- isRealTerminal -----------------------------------------------------------------------------
// What    : Tells if the output is a real terminal (not a pipe or a file).
// Returns : true for a terminal
// Used by : main
//------------------------------------------------------------------------------------------------
bool isRealTerminal() {
#ifdef _WIN32
    return _isatty(_fileno(stdin)) && _isatty(_fileno(stdout));
#else
    return isatty(STDIN_FILENO) && isatty(STDOUT_FILENO);
#endif
}

//--- main ---------------------------------------------------------------------------------------
// What    : Program entry. Reads --plain and --color, sets the color mode, and runs the store. In a
//           real terminal the screen-clearing filters are installed around runStore; with piped
//           input or output nothing is cleared.
// Returns : 0
// Used by : the operating system (program entry point)
//------------------------------------------------------------------------------------------------
int main(int argc, char* argv[]) {
    bool allowColor = true;
    bool forceColor = false;
    bool forceClear = false;
    bool showcase = false;
    for (int i = 1; i < argc; i++) {
        std::string arg = argv[i];
        if (arg == "--plain") allowColor = false;
        if (arg == "--color") forceColor = true;
        if (arg == "--clear") forceClear = true;
        if (arg == "--showcase") showcase = true;
    }
    g_floodBlankLines = forceClear;
    tuiInit(allowColor, forceColor);

    if (!isRealTerminal() && !forceClear) {
        runStore(showcase);                         // output to a file or piped input: no clearing
        return 0;
```

```
    }

    std::streambuf* originalOut = std::cout.rdbuf();
    std::streambuf* originalIn = std::cin.rdbuf();
    ScreenBuf screen(originalOut);
    LineBuf input(originalIn, &screen);
    std::cout.rdbuf(&screen);
    std::cin.rdbuf(&input);

    screen.clearNow();
    runStore(showcase);

    screen.finish();
    std::cout.rdbuf(originalOut);
    std::cin.rdbuf(originalIn);
    return 0;
}
```

Note: *Due to the extensive number of header dependencies within the project architecture, header file source listings are excluded from this document. Complete class declarations and interface definitions can be referenced directly in the Github repository.*

A.1 main.cpp

A.2 [file_name.h / file_name.cpp]

A.3 [timing / benchmark code]

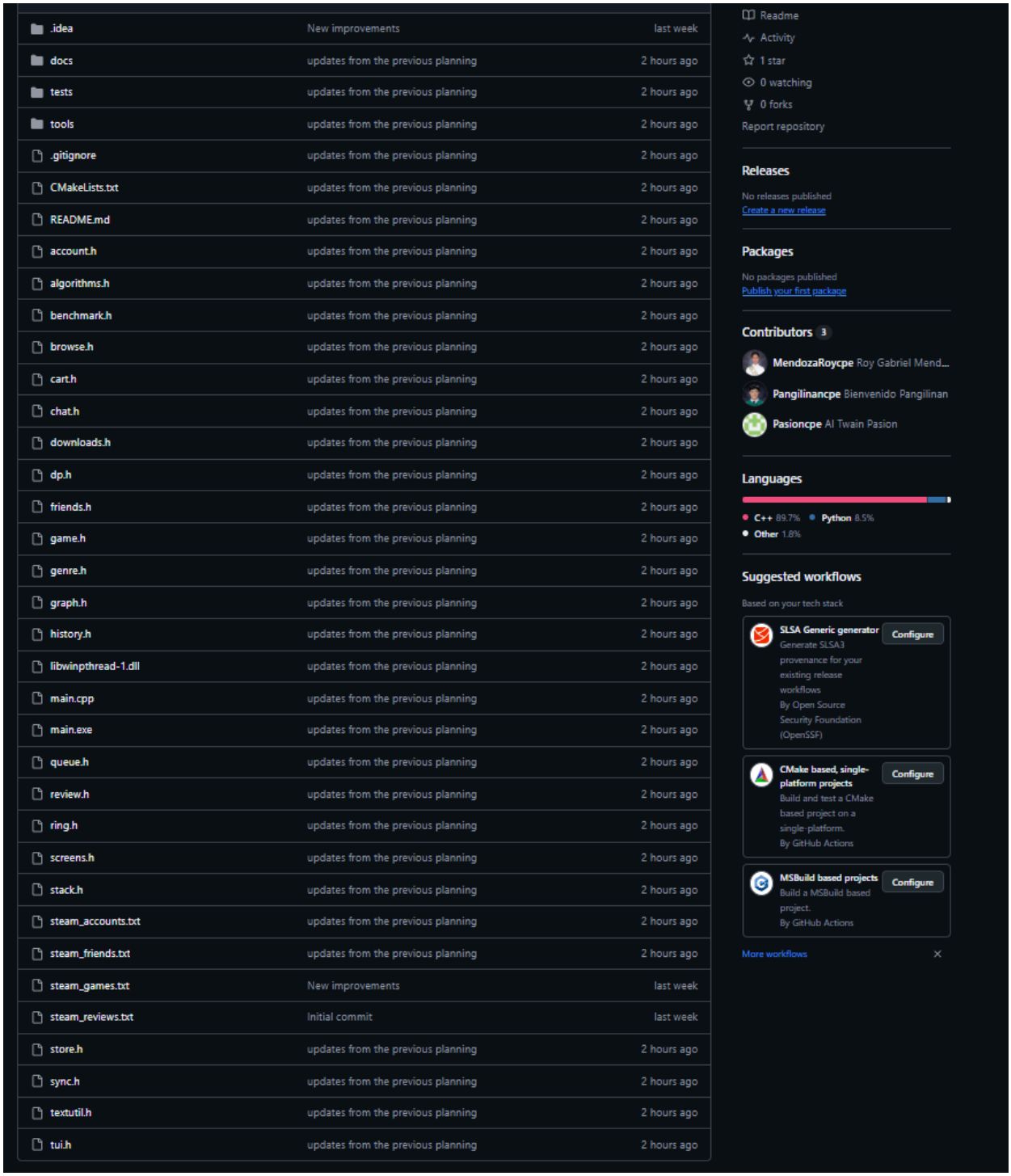
Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	% Total of The work overall
1	Mendoza, Roy Gabriel I.	2513234	Project Leader, Code Project Outliner	Section 1 and 5 of the document, tui.h, game.h, graph.h chat.h	[33.33.]
2	Pangilinan, Bienvenido B.	2511157	Code, Documenter	Reworked main.cpp into a terminal-based interface, Section 2 documentation, flowchart	[18]
3	Pasion, Al Twain D.	2516562	Code troubleshooting and Analysis	Account.h, algorithms.h, benchmark.h, browse.h cart.h, downloads.h dp.h, friends.h game.h	[20]

4	Reyes, Eli Justin L.	2512536	Code Documenter	Purposes and Locations of functions, friends.h, genre.h, and Code Documentation.	[14.33 5]
5	Torres, Carl Andrei D.	2512311	Code Documenter	Scope and Limitations, Vector Algorithms Documentation, Outputs, and store.h	[14.33 5]

Appendix C. GitHub Repository

Item	Details
Repository link	https://github.com/Pangilinancpe/Final-Project---DSA
Branch / release submitted	Main
README contents	Project description, features, system requirements, compilation and execution instructions, sample interface, project structure, data structures and algorithms applied, testing procedures, and team members.
How to compile	g++ -std=c++17 -pthread main.cpp -o game_store



.idea	New improvements	last week
docs	updates from the previous planning	2 hours ago
tests	updates from the previous planning	2 hours ago
tools	updates from the previous planning	2 hours ago
.gitignore	updates from the previous planning	2 hours ago
CMakeLists.txt	updates from the previous planning	2 hours ago
README.md	updates from the previous planning	2 hours ago
account.h	updates from the previous planning	2 hours ago
algorithms.h	updates from the previous planning	2 hours ago
benchmark.h	updates from the previous planning	2 hours ago
browse.h	updates from the previous planning	2 hours ago
cart.h	updates from the previous planning	2 hours ago
chat.h	updates from the previous planning	2 hours ago
downloads.h	updates from the previous planning	2 hours ago
dp.h	updates from the previous planning	2 hours ago
friends.h	updates from the previous planning	2 hours ago
game.h	updates from the previous planning	2 hours ago
genre.h	updates from the previous planning	2 hours ago
graph.h	updates from the previous planning	2 hours ago
history.h	updates from the previous planning	2 hours ago
libwinpthread-1.dll	updates from the previous planning	2 hours ago
main.cpp	updates from the previous planning	2 hours ago
main.exe	updates from the previous planning	2 hours ago
queue.h	updates from the previous planning	2 hours ago
review.h	updates from the previous planning	2 hours ago
ring.h	updates from the previous planning	2 hours ago
screens.h	updates from the previous planning	2 hours ago
stack.h	updates from the previous planning	2 hours ago
steam_accounts.txt	updates from the previous planning	2 hours ago
steam_friends.txt	updates from the previous planning	2 hours ago
steam_games.txt	New improvements	last week
steam_reviews.txt	Initial commit	last week
store.h	updates from the previous planning	2 hours ago
sync.h	updates from the previous planning	2 hours ago
textutil.h	updates from the previous planning	2 hours ago
tui.h	updates from the previous planning	2 hours ago

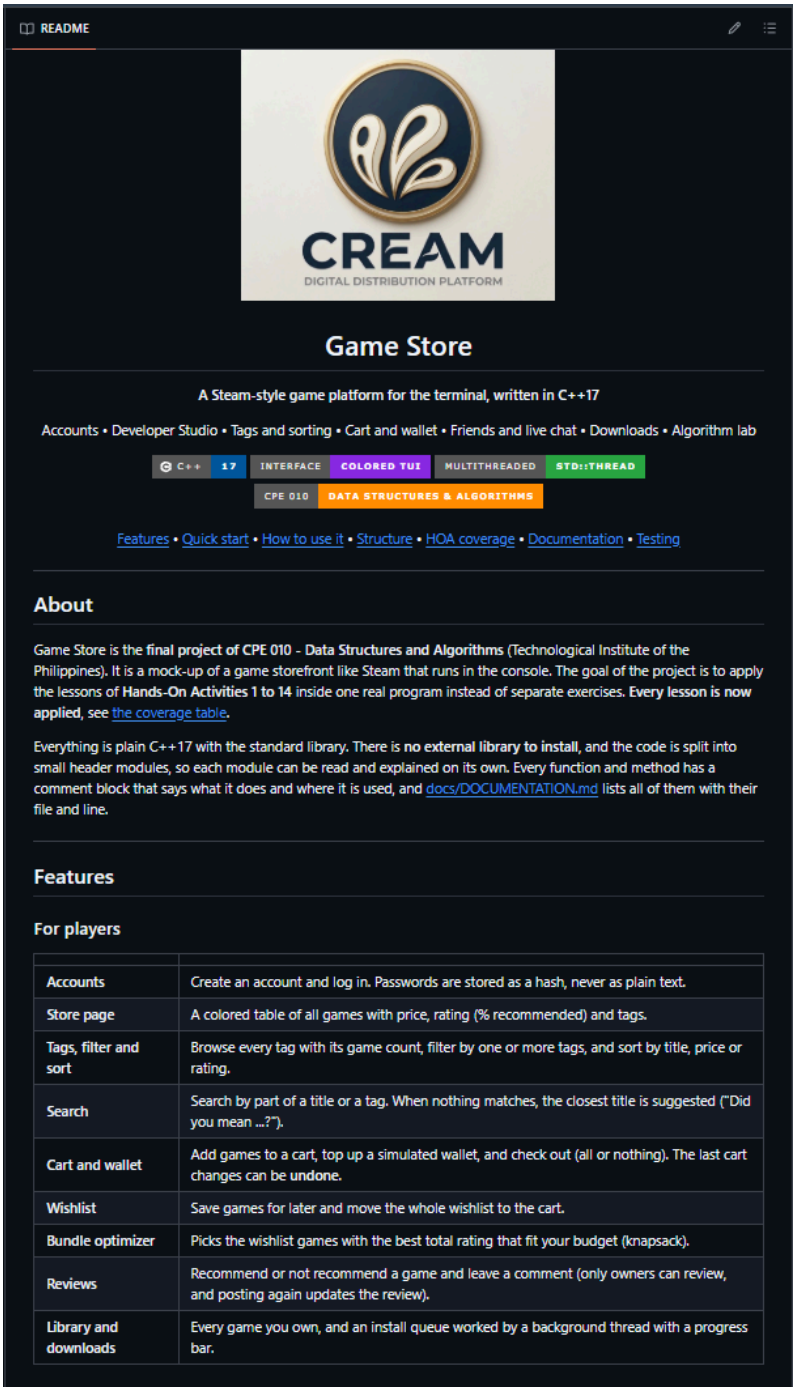


Figure 6.1 Screenshot of the repository page showing files

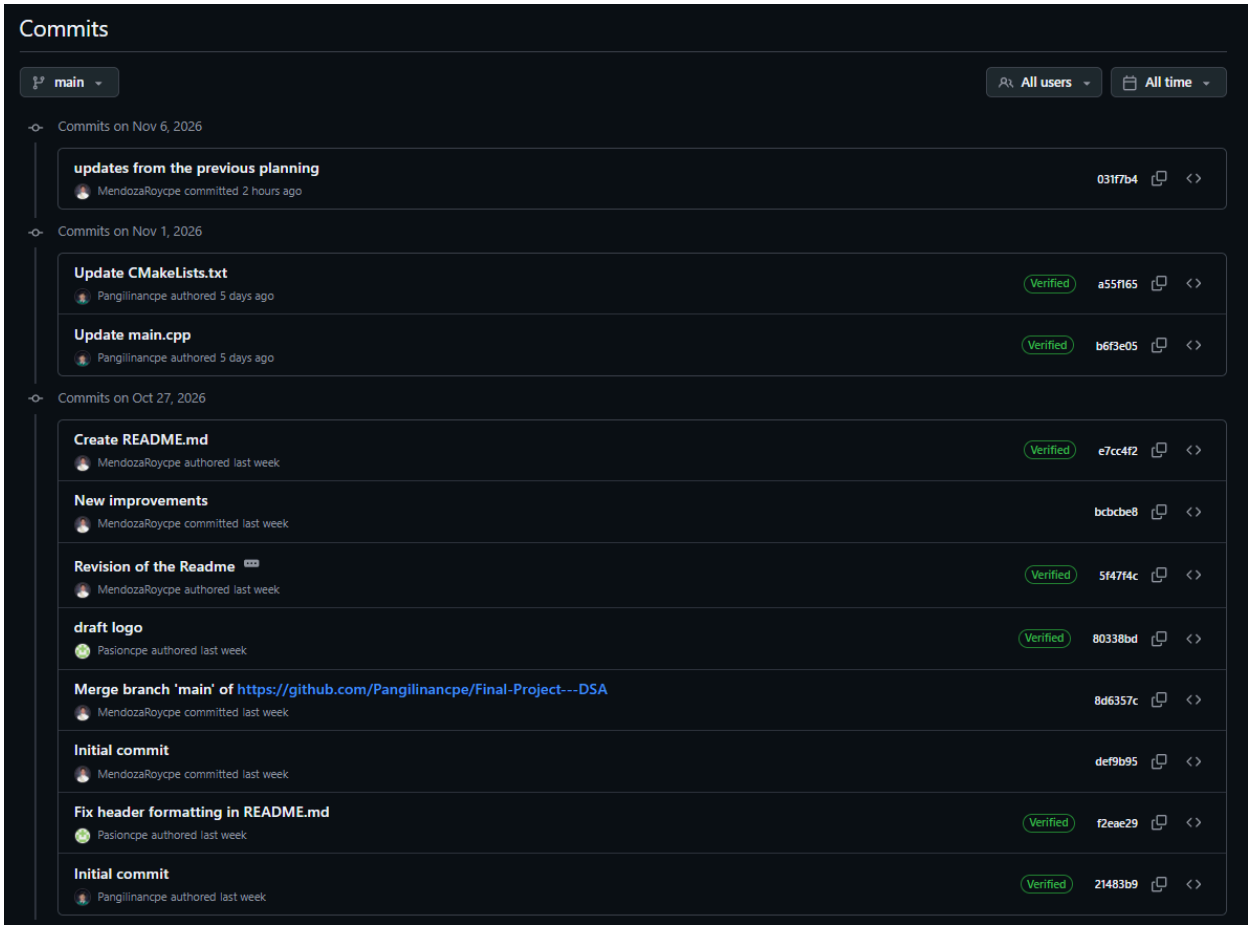


Figure 6.2 Screenshot of the commit history

EVALUATION CRITERIA

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15
Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100